Addis Ababa University — School of Information Technology and Engineering
Software Testing and Validation — Final Project

Test Summary Report

Clinic Booking System

Student	Biruk Mulatu
Student ID	[STUDENT ID — fill in before submission]
Instructor	Abel Tadesse
Date	4 September 2026 (revised 6 September 2026)

1. What was tested

- The four course rules (fee by age, booking eligibility, appointment lifecycle, Rule 3b late-cancellation guard) via unit tests derived from equivalence partitioning, boundary value analysis, a decision table and state transition testing — full derivation in the test design document, §1–4.
- Seven extension rules (F–L: patient registration, doctor load, 24-hour reminder, insurance coverage, no-show suspension, reschedule, waitlist-offer expiry), each a pure `core/` function tested with the same techniques (test design document §5).
- The booking outcome by **pairwise (all-pairs) testing** over its six independent factors — a fifth technique (`BookingPairwiseIT`, 11 cases).
- **Mutation testing** (PIT) of `et.aau.clinic.core`: 165 injected mutations, 165 killed — a 100% mutation score confirming the tests detect faults, not just execute lines.
- The `AppointmentService` orchestration layer against Mockito-mocked repositories, a `Clock` stub and a mocked `NotificationService`; and the same service wired to real Spring Data JPA repositories and a real H2 database, including a Mockito spy on the real notification service.
- Concurrent booking of one slot under real threads (`BookingConcurrencyIT`, 8 contending threads) — the case DEF-005 fixed.
- Four end-to-end journeys through a real headless Chrome browser via Selenium and the Page Object pattern (`BookingJourneyIT`): the required log in → view slots → book → see fee → cancel journey, plus insurance coverage on confirmation, no-show suspension blocking a booking, and reschedule.
- Continuous integration itself: both the GitHub Actions workflow and a Jenkins pipeline (run via the provided Docker setup) were exercised end-to-end; both now also run mutation testing as a distinct step.
- Regression detection: a deliberate one-line defect was introduced, observed failing in CI, and fixed (defect log §3).

Level	Test classes	Test methods
Unit	17 classes — the four course-rule tests plus `PatientRegistrationPolicyTest`, `DoctorLoadTest`, `ReminderPolicyTest`, `CoverageCalculatorTest`, `SuspensionPolicyTest`, `ReschedulePolicyTest`, `WaitlistOfferPolicyTest`, `AppointmentServiceTest`, and the Phase C–E tests (`QueueStateMachineTest`, `VisitRecordPolicyTest`, `AvailabilityExpanderTest`, `SlotReconcilerTest`, `QueueServiceTest`, `VisitRecordServiceTest`)	228
Integration	10 `*IT` classes — `AppointmentServiceIT`, `AppointmentJourneyIT`, `BookingPairwiseIT`, `BookingConcurrencyIT`, `AdminManagementIT`, `AdminCrudIT`, `RoleAccessIT`, `QueueServiceIT`, `VisitRecordServiceIT`, `VisitRecordApiIT`	72
System	`BookingJourneyIT` (Page Objects: `LoginPage`, `SlotsPage`, `BookingPage`, `ConfirmationPage`, `MyAppointmentsPage`, plus a shared `AbstractPage`)	5
Total		**305**

228 : 72 : 5 is a genuine pyramid shape — the large majority of cases run fast and without a container or browser, with progressively fewer, slower, broader-scope tests above them. Mutation testing runs on top of the unit level as a separate quality gate.

2. What was not tested

- **Real SMS delivery.** By design, `NotificationService` has only a logging implementation; there is no real gateway to test against.
- **Non-functional testing** — load, performance under many concurrent users, accessibility, and security penetration testing are all out of scope per the test plan (§1.2). Note the one concurrency case that *is* covered (`BookingConcurrencyIT`) is a correctness test of the slot lock, not a load or performance test.
- **The React front end** (`frontend/`) — a decorative alternative UI over the same JSON API — has no automated tests of its own and is not part of the graded suite. The Thymeleaf pages are what Selenium drives.
- **The JSON API controllers and DTOs under** `web/api/` have no dedicated unit tests; they are thin pass-throughs to `AppointmentService` (which is fully tested) and are exercised indirectly by the MockMvc integration tests. This is the dominant reason whole-application branch

coverage is 72.5% rather than higher — see doc 3 §2. A few genuinely defensive branches elsewhere (a catch-all in `GlobalExceptionHandler`, some getters never read by a test) make up a small remainder. None of that uncovered code is inside a business rule; `core/` is at 100% branch and 100% mutation score.

3. Results against exit criteria

Against the exit criteria set in the test plan (§3.2):

Criterion	Result
`mvn clean verify` passes in one run (unit + integration + system)	**Met — BUILD SUCCESS, 305/305 tests green**
≥80% branch coverage on `et.aau.clinic.core`, gate enforced	**Met — 100% (134/134 branches)**
≥90% mutation score on `et.aau.clinic.core`, PIT gate enforced	**Met — 100% (165/165 mutants killed)**
Every valid/invalid state transition in the 7×7 grid has an explicit test	**Met — 10 valid tests + a generated, count-asserted set of all 39 invalid pairs**
Every specified boundary value has an explicit test	**Met — the fee boundaries (–1, 0, 17, 18, 64, 65, 120, 121); the 2h notice boundary; the 24h cancellation boundary; and the boundaries of rules F–L (password length and age; doctor limit; 24h reminder window; 0/100% coverage; count-of-3 and 90-day window; 2h reschedule notice; 2h offer window)**
GitHub Actions green on `main`; Jenkins run at least once via Docker	**Met — both verified end-to-end with real build attempts, not just reviewed configuration; both extended with a mutation-testing step**
A deliberate regression introduced, caught by CI, then fixed, with evidence recorded	**Met — see `docs/regression-demo.md` and defect log §3**
All defects logged are Closed, or explicitly carried as residual risk	**Met — DEF-001 to DEF-005 all Closed and verified**

4. Outstanding defects and residual risk

Item	Status	Residual risk
DEF-001 to DEF-005	All Closed, verified	None outstanding — each has a verification step confirming the fix under the same conditions that exposed it. DEF-005 in particular is guarded by a test proven to fail when the fix is reverted.
Concurrent double-booking of a slot	**Fixed (DEF-005)** — pessimistic row lock on the slot + `@Transactional` on the three booking paths; guarded by `BookingConcurrencyIT`	**Low.** The check-then-insert section is now serialised. Residual: the lock relies on the single H2 datasource; a multi-instance deployment would additionally need the database's own constraint, which is noted as future work but not required for this single-process application.
The `web/api/` JSON controllers have no dedicated unit tests	Accepted — additive, ungraded code	**Low.** Each endpoint is a thin delegate to the fully-tested `AppointmentService` and is exercised indirectly by MockMvc integration tests; the logic that could actually be wrong lives in the service, not the controller.
Defect metrics based on a small sample (5 defects, 1 developer, no external review yet)	N/A	**Low–Medium.** DRE of 100% and 0 escaped defects reflect that no external party has evaluated the system yet, not that it is defect-free; the instructor's evaluation is the first real external validation this project will undergo (see §6's verification/validation distinction).

5. Release recommendation

Recommendation: ready to submit.

All exit criteria in §3 are met. On the graded package the branch-coverage gate is exceeded (100% vs. an 80% target) and a second,

stronger gate — a 100% mutation score — confirms the tests detect faults rather than merely execute lines. All five defects found during development are closed and verified, including DEF-005, which closes the concurrent-double-booking gap that an earlier version of this report had carried as a known limitation. The regression-detection requirement has real, reproducible evidence (a failing CI run and a passing one) rather than a description of what would happen.

There is no known open functional gap. The residual risks in §4 are all "low" and are about the limits of a single-developer, single-process, not-yet-externally-reviewed project rather than about suspected defects in the code. For a course project whose explicit grading criterion is the testing effort, and equally for the application's own correctness, this is a defensible stopping point.

6. Foundations reflection — error, fault, failure; verification vs. validation

The defect: DEF-005, the concurrent double-booking of a slot (full write-up in the defect log). Two patients booking the same free slot at the same instant could both succeed, leaving two active appointments for one slot.

- **Error** — the human mistake was treating a check followed by an insert as if it were one atomic step. When `requestBooking` was written it read naturally as "if the slot is free, create the appointment", and that reading quietly assumes nothing else changes the slot's state in between. Nothing in a single-threaded test ever challenges that assumption.
- **Fault** — the resulting defect in the code: the "is the slot free?" query (`existsBySlotAndStatusIn`) and the `save` that followed it were two separate statements with no lock spanning them, and the method was not transactional. Two callers could interleave: both read "free", both insert. The fault sat latent in the code from the first version of the booking path.
- **Failure** — the observable symptom: under eight threads released together, 2–4 of them were approved and the database ended up with several `REQUESTED` rows for one slot — a violation of the system's core invariant, one active appointment per slot. The same fault produced no failure at all in any sequential test, which is why it survived hundreds of passing runs before a concurrency test was written to expose it.

Verification or validation? This was caught by **verification**. The invariant it violates — one active appointment per slot — is part of the system's own specification (it is what Rule 2's condition C1 exists to enforce). The concurrency test checks the code against that specification, with no real user or stakeholder involved: it is a machine asserting an internal correctness property, which is exactly what "are we building the product right" means. Validation — checking with an actual user that a booking system that never double-books is the *right* product to build — was never in question and played no part in finding this. The wider lesson is that verification is only as good as the conditions it exercises: the specification implied concurrency mattered, but no test probed it until one was deliberately added, and the fault was invisible until then.